TD 4 – TP télécommande - Gestion des Lampes

Objectif pédagogique

A l'issue de ce TP, vous devrez

- savoir faire un diagramme de classe élémentaire
- faire le lien entre ce diagramme de classe et son implémentation

Rendu

Votre travail devra impérativement se trouver dans le nouveau dépôt git `2026_coo_domotique_`*`login1_login2`*.

1 Mise en place du dépôt git (10 min)

Question 1

Avec votre binôme, créez un seul dépôt git nommé `2026_coo_domotique_`*`login1_login2`*.

Question 2

Faire des clones. Un des membres du binôme doit ajouter un fichier `README.txt` contenant les noms et prénoms des membres du binôme. L'autre membre du binôme récupère ce fichier en faisant un `pull` sur le dépôt distant.

Vous penserez à faire des `push` réguliers au cours du TP pour limiter les conflits et pour que le dépôt distant soit toujours à jour.

Question 3

Un des membres du binôme crée dans le dépôt un seul projet IntelliJ `domotique`, l'autre membre le récupère avec un `pull`.

Attention :
Veillez à chaque fois à ce que la création du fichier ne se fasse que par un seul membre du binôme: si deux fichiers de même noms sont créés simultanément, cela risque de générer des conflits.

2 Descriptif du sujet

2.1 Description générale

Dans les TPs suivants, vous allez être amenés à développer une télécommande universelle capable de gérer plusieurs appareils.

Ce premier TP constitue la mise en place des premiers éléments et se poursuivra dans les TP suivant en augmentant en complexité. Cette partie se limitera à développer une télécommande capable d'allumer et d'éteindre des lampes.

Attention :
Respectez la démarche de développement présentée dans la partie 2 et n'écrivez du code Java que lorsque le sujet vous le demande.

2.2 Sujet de Manipulation de Lampes (rappel)

Dans le TD précédent, vous avez dessiné un diagramme de classe qui rend compte des éléments rappelés ci-dessous. Ce diagramme de classe sera la base du travail aux question suivantes.

Pour rappel, cette première partie du sujet consiste à développer deux classes:
- une classe `Lampe` représentant une lampe;
- une classe `Telecommande` chargée de commander plusieurs lampes à distance.

La classe `Lampe` possède:
- deux attributs
  - ☐ un attribut `nom` permettant d'identifier la lampe;
  - ☐ un attribut `allume` de type `boolean` valant `True` si et seulement si la lampe est allumée;
- un constructeur qui prend en paramètre le nom de la `Lampe` et construit une lampe éteinte;
- deux méthodes `allumer` et `eteindre` qui changent l'état de la lampe.

La classe `Telecommande` possède:
- un attribut `lampes` de type liste de `Lampe` correspondant aux lampes contrôlées par la télécommande;
- un constructeur vide qui construit une télécommande ne contrôlant initialement aucune lampe;
- une méthode `ajouterLampe` qui prend une `Lampe` en paramètre et qui ajoute un lien dans la télécommande permettant de commander cette lampe;

- des méthodes `activerLampe` et `desactiverLampe` qui prennent un entier `indiceLampe` en paramètre et effectuent l'opération correspondante sur la lampe de la télécommande à l'indice `indiceLampe`;
- une méthode `activerTout()` qui active toutes les lampes contrôlées par la télécommande.

Enfin, ces deux classes possèdent des méthodes `toString` retournant leurs descriptions. Pour la `Telecommande`, vous afficherez les lampes précédées des indices où elles se situent dans la liste.

2.3 Diagramme de classe (10 min)

Reprenez le diagramme de classe du TD précédent qui rend compte de l'organisation de l'application.

Question 4

Corriger si besoin ce diagramme suite aux remarques de votre enseignant et ajoutez le fichier image dans votre dépôt.

3 Manipulation via `Telecommande`

3.1 Intégration classe `LampeTest` et `Lampe` (5 min)

La classe `Lampe` et la classe de test `LampeTest` vous sont fournies sous Arche.

Question 5

Un seul membre du dépôt intègre dans votre projet les classes `Lampe` et `LampeTest`, puis, synchroniser les dépôts avec des `push` et `pull`.

3.2 Squelette de la classe `Telecommande` (10 min)

Afin d'écrire les tests de la classe `Telecommande`, il faut d'abord écrire une classe `Telecommande` vide qui compile.

Pour être sûr que cette classe compile et lève des erreurs lors des appels des méthodes qui n'ont pas été écrites, vous allez:

- créer la classe `Telecommande` vide;
- écrire les signatures des méthodes de la classe `Telecommande`;
- dans chaque méthode lever une erreur avec l'instruction ci-dessous.

```
throw new Error("code non ecrit");
```

Un objet `Error` hérite de `Throwable` (ce qui permet d'utiliser `throw`) mais n'est pas une `Exception`. Les objets `Error` ont pour objectif de faire arrêter le programme en cas d'erreur détectée (et ne doivent donc pas être "catchés"). Les `Error` n'ont pas besoin d'être signalés par un `throws` et remontent donc les méthodes jusqu'à la sortie du `main` (ou du test).

Question 6

Écrire le squelette de la classe `Telecommande`.

3.3 Classe de test de `Telecommande` (15 min)

Prenez le temps de lire comment la classe de test de la classe `Lampe` est écrite.

Pour chaque méthode de test, la classe de test possède:

1. Une entête qui décrit la nature du test
   - le nom de la méthode testée;
   - le cas correspondant au test.
2. Une structuration particulière en trois blocs annoncés par des commentaires
   - un bloc de préparation des données;
   - un bloc exécutant la méthode de test;
   - un bloc vérifiant les résultats obtenus.

Vous êtes amenés à respecter cette structure lors de l'écriture de vos tests.

Partagez-vous les tests à écrire entre membres du binôme si vous avez accès à plusieurs machines au cours du TP.

Question 7

Écrire des tests unitaires simples sur la classe `Telecommande`:

- ajout d'une lampe à une télécommande vide;
- ajout d'une lampe à une télécommande avec 1 élément;
- activation d'une lampe existante en position 0;
- activation d'une lampe existante en position 1;
- activation d'une lampe inexistante.

3.4 Écriture du code (15 min)

Question 8

Une fois les tests écrits, écrire les contenus des méthodes de la classe `Telecommande` en prenant en compte les cas exceptionnels auxquels vous avez pensé.

La classe `Main` fournie dans Arche permet de lancer un menu avec une télécommande liée à quatre lampes et permet d'interagir avec les lampes.

Question 9

Compléter la classe jusqu'à ce que tous les tests passent et ajouter un tag `v1.0` sur cette version de l'application.

4 Manipulation de chaines hifi (30 min)

En plus de manipuler des lampes, on souhaite que la télécommande puisse manipuler des chaines hifi.

Une chaine hifi est décrite par la classe `Hifi` fournie sur Arche et présentée ci-dessous. Cette classe possède un attribut entier son pouvant aller de 0 à 100.

```
/**
 * classe qui represente une chaine hifi
 */
class Hifi
{

  /**
   * l'intensite du son de la chaine
   * on suppose que la chaine est eteinte si le son vaut 0
   */
  int son=0;


  /**
   * permet d'allumer ou d'augmenter le son de la chaine
   */
  public void allumer() {
    this.son+=10;
    //son maximum
    if (this.son>100)
      this.son=100;
  }

  /**
   * permet d'eteindre la chaine (son mis a 0)
   */
  public void eteindre() {
    this.son=0;

  }

  /**
   * surcharge de la methode affichage
   * sous la forme "Hifi:10"
   */
  public String toString()
  {
```

```
    String r="";
    r+="Hifi:"+son;
    return(r);
  }
}
```

Question 10

Modifier la classe `Telecommande` pour manipuler des chaines Hifi en plus des lampes.

Rendu

Pour la prochaine séance, finir l'ajout de l'utilisation de la classe `Hifi` dans `Telecommande` et ajouter le tag `v2`.